

Fișă de lucru – Metode suplimentare de prelucrare și multiplicare a listelor în Python

Noțiuni teoretice extinse

1. Ce sunt metodele suplimentare și cum funcționează?

- **Metode de clasă (methods):** Spre deosebire de funcțiile globale, metodele sunt funcții speciale atașate direct obiectului de tip listă. Ele se apelează obligatoriu folosind sintaxa cu punct: `nume_lista.metoda()`. Majoritatea acestor metode modifică direct structura în memorie (acțiune de tip *in-place*), fără a crea o listă nouă.

2. Instrumente avansate de modificare și clonare

- **.extend()** — Metodă utilizată pentru extinderea unei liste prin adăugarea tuturor elementelor dintr-o altă colecție (o altă listă) la sfârșitul acesteia. Spre deosebire de `.append()`, care adaugă o listă ca pe un singur element compact, `.extend()` „despachetează” valorile și le concatenează liniar.
 - *Exemplu:* Dacă avem `listă_stoc_baza = [10, 20]` și `listă_stoc_nou = [30, 40]`, apelul `listă_stoc_baza.extend(listă_stoc_nou)` va modifica lista inițială, aceasta devenind `[10, 20, 30, 40]`.
- **.reverse()** — Metodă utilizată pentru inversarea completă a ordinii elementelor dintr-o listă direct în memoria calculatorului. Elementele sunt oglindite de la coadă la cap.
 - **⚠ Atenție:** Această metodă nu sortează elementele numeric sau alfabetic, ci doar le schimbă pozițiile (indecșii) în mod invers față de starea inițială.
 - *Exemplu:* Pentru `listă_coduri = [5, 12, 8, 20]`, comanda `listă_coduri.reverse()` va transforma lista direct în `[20, 8, 12, 5]`.
- **.clear()** — Metodă utilizată pentru ștergerea instantanee și integrală a tuturor elementelor din interiorul unei liste.
 - *Exemplu:* Pentru `listă_eroari_sistem = [404, 500, 403]`, apelul `listă_eroari_sistem.clear()` va goli complet structura, aceasta devenind o listă goală, adică `[]`. Obiectul rămâne valid în memorie, dar va avea lungimea 0.
- **.remove()** — Metodă utilizată pentru eliminarea din listă a unui element pe bază de valoare. Ea caută elementul de la stânga la dreapta și șterge prima apariție a acestuia.
 - **⚠ Atenție:** Dacă valoarea căutată nu se află în listă, Python va genera o eroare de tipul `ValueError`. După ștergere, elementele din dreapta se deplasează automat la stânga pentru a umple golul, reindexând lista.
 - *Exemplu:* Pentru `listă_fruite = ["măr", "pară", "măr"]`, comanda `listă_fruite.remove("măr")` va șterge doar primul element, lista devenind `["pară", "măr"]`.
- **.copy()** — Metodă utilizată pentru crearea unei copii independente (o clonă) a listei existente într-o zonă nouă de memorie.
 - *De ce este necesară?* Dacă scriem simplu `listă_2 = listă_1`, ambele variabile vor indica exact aceeași adresă de memorie (efectul de *aliasing*). Modificând `listă_2`, se va modifica automat și `listă_1`. Utilizarea `.copy()` rupe această legătură, creând un obiect nou.
 - *Exemplu:* Dacă avem `date_initiale = [1, 2]` și executăm `date_backup = date_initiale.copy()`, se generează o listă nouă independentă. Orice modificare ulterioară asupra variabilei `date_backup` nu va afecta valorile memorate în `date_initiale`.

Activitate practică:

Scenariu: „Misiunea Administratorului de Sistem la o Agenție Spațială – Faza II (Procesarea avansată a datelor)”

Context: Ești programatorul principal la centrul de control al zborurilor spațiale. Serverul central a recepționat două fluxuri independente de date telemetrice ce conțin coduri de eroare și identificatori numerici. Sistemul automat are nevoie ca aceste liste să fie concatenate, copiate în siguranță pentru backup, inversate ca prioritate, curățate de elemente parazite și, în final, resetate complet pentru a pregăti următoarea lansare.

✂ Cerințe :

Implementați în Python următoarele:

Pasul 1: Inițializarea listelor de telemetrie Declarați o listă principală numită `telemetrie_baza` care conține elementele în următoarea ordine: numărul întreg 101, numărul întreg 104 și numărul întreg 108. Declarați o a doua listă numită `telemetrie_supliment` care conține numerele întregi 202 și 205.

Pasul 2: Fuzionarea listelor prin extindere Pentru a centraliza toate datele într-o singură structură continuă, adăugați toate elementele din lista secundară la sfârșitul listei principale, utilizând metoda care desface colecțiile. Afișați lista `telemetrie_baza` pe ecran pentru a verifica unificarea datelor.

Cod sugestiv: `telemetrie_baza.extend(...)`

Pasul 3: Crearea unui Backup de Siguranță (Clonarea) Înainte de a procesa datele brute, inspectorii cer salvarea stării actuale. Creați o copie complet independentă în memorie a listei principale într-o variabilă nouă numită `telemetrie_backup`, utilizând metoda de clonare. Afișați lista de backup pe ecran.

Cod sugestiv: `telemetrie_backup = telemetrie_baza.copy()`

Pasul 4: Inversarea priorității elementelor Racheta solicită citirea datelor în ordine inversă față de momentul recepției (de la coadă la cap). Modificați așezarea elementelor direct în memorie folosind metoda de inversare structurală și afișați lista pe ecran.

Cod sugestiv: `telemetrie_baza.reverse()`

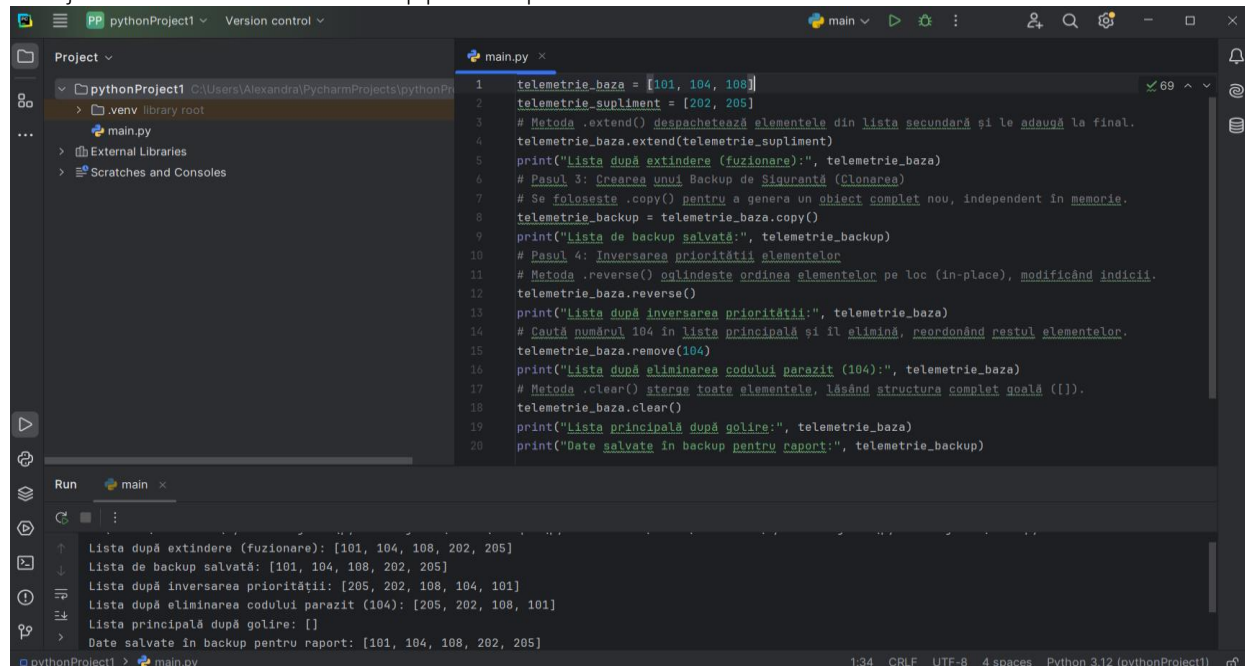
Pasul 5: Eliminarea codului parazit pe bază de valoare Sensorii au detectat că valoarea 104 reprezintă o eroare de comunicare falsă. Eliminați din lista principală inversată acest cod parazit utilizând metoda potrivită bazată pe valoare. Afișați lista pe ecran pentru a verifica eliminarea.

Cod sugestiv: `telemetrie_baza.remove(...)`

Pasul 6: Resetarea memoriei tampon (Golirea listei) Misiunea de citire s-a încheiat cu succes, iar datele au fost transmise satelitului. Goliți instantaneu tot conținutul listei principale `telemetrie_baza` pentru a elibera memoria consolei. Afișați lista pe ecran pentru a demonstra că a devenit o listă goală.

Cod sugestiv: `telemetrie_baza.clear()`

Pasul 7: Verificarea integrității raportului final Pentru a demonstra că sistemul de backup a funcționat corect și nu a fost afectat de modificările ulterioare, afișați pe ecran conținutul listei `telemetrie_backup` precedat de mesajul exact: "Date salvate în backup pentru raport:".



```
1 telemetrie_baza = [101, 104, 108]
2 telemetrie_supliment = [202, 205]
3 # Metoda .extend() despachetează elementele din lista secundară și le adaugă la final.
4 telemetrie_baza.extend(telemetrie_supliment)
5 print("Lista după extindere (fuzionare):", telemetrie_baza)
6 # Pasul 3: Crearea unui Backup de Siguranță (Clonarea)
7 # Se folosește .copy() pentru a genera un obiect complet nou, independent în memorie.
8 telemetrie_backup = telemetrie_baza.copy()
9 print("Lista de backup salvată:", telemetrie_backup)
10 # Pasul 4: Inversarea priorității elementelor
11 # Metoda .reverse() oglindește ordinea elementelor pe loc (in-place), modificând indicii.
12 telemetrie_baza.reverse()
13 print("Lista după inversarea priorității:", telemetrie_baza)
14 # Caută numărul 104 în lista principală și îl elimină, reordonând restul elementelor.
15 telemetrie_baza.remove(104)
16 print("Lista după eliminarea codului parazit (104):", telemetrie_baza)
17 # Metoda .clear() șterge toate elementele, lăsând structura complet goală ([]).
18 telemetrie_baza.clear()
19 print("Lista principală după golire:", telemetrie_baza)
20 print("Date salvate în backup pentru raport:", telemetrie_backup)
```

Run main

```
↑ Lista după extindere (fuzionare): [101, 104, 108, 202, 205]
↓ Lista de backup salvată: [101, 104, 108, 202, 205]
↑ Lista după inversarea priorității: [205, 202, 108, 104, 101]
↓ Lista după eliminarea codului parazit (104): [205, 202, 108, 101]
↑ Lista principală după golire: []
↓ Date salvate în backup pentru raport: [101, 104, 108, 202, 205]
```

Problemă rezolvată și explicată pas cu pas

Enunț: Se dau inițial două liste distincte care stochează datele de inventar ale unui depozit de tehnică: o listă principală eterogenă numită `inventar_baza = ["Echipament", 50, 120, 35]` și o listă secundară pur numerică numită `inventar_nou = [80, 15]`. Să se scrie un program în limbajul Python care realizează următoarele prelucrări succesive în memorie, demonstrând proprietățile metodelor suplimentare ale clasei `list`:

1. Elimină elementul de tip șir de caractere ("Echipament") de la începutul colecției pentru ca lista principală să devină pur numerică.
2. Extinde lista `inventar_baza` prin adăugarea tuturor elementelor din lista `inventar_nou` dintr-o singură comandă.
3. Creează o copie independentă (clonă) a listei principale extinse într-o variabilă nouă numită `inventar_istoric`.
4. Inversează ordinea tuturor elementelor din lista `inventar_baza` direct în memorie.
5. Elimină valoarea numerică 120 din lista principală modificată, utilizând metoda bazată pe valoare.
6. Șterge integral toate elementele din lista `inventar_baza` pentru a finaliza sesiunea de lucru și afișează starea finală a ambelor liste.

3. Exemplu

- **Date de intrare (listele inițiale):** `inventar_baza = ["Echipament", 50, 120, 35]`
`inventar_nou = [80, 15]`
- **Date de ieșire (afișate pe ecran):** o Lista principală după golire totală: []
o Copia de siguranță (`inventar_istoric`): [50, 120, 35, 80, 15]

4. Analiza exemplului

Pentru a înțelege cum se ajunge la rezultatele afișate, analizăm starea listei în memorie după fiecare instrucțiune obligatorie din enunț:

- **Starea inițială:** `inventar_baza = ["Echipament", 50, 120, 35]` (Colecție eterogenă cu text și numere, lungime inițială 4) și `inventar_nou = [80, 15]` (lungime 2).
- **Operația 1 (Eliminare text):** Se șterge elementul "Echipament". Lista devine pur numerică: [50, 120, 35]. Toate elementele rămase se reindexează automat la stânga începând cu poziția 0.
- **Operația 2 (Extinderea listei):** Se apelează metoda `.extend()`. Elementele din `inventar_nou` sunt descătușate și concatenate la finalul listei principale. `inventar_baza` devine: [50, 120, 35, 80, 15].
- **Operația 3 (Copiere / Clonare):** Se utilizează `.copy()` pentru a genera un obiect complet nou la o altă adresă de memorie. Variabila `inventar_istoric` memorează starea curentă: [50, 120, 35, 80, 15]. Această listă este acum independentă.
- **Operația 4 (Inversare structurală):** Se apelează `.reverse()`. Elementele listei principale se oglindesc direct în memorie de la coadă la cap. Lista devine: [15, 80, 35, 120, 50].
- **Operația 5 (Eliminare pe bază de valoare):** Se execută ștergerea valorii 120. Sistemul caută numărul în lista inversată, îl elimină, iar elementul următor (50) se reindexează la stânga. Lista devine: [15, 80, 35, 50].
- **Operația 6 (Curățare totală):** Se apelează `.clear()` pe lista principală. Toate valorile sunt eliminate instantaneu din structură. `inventar_baza` devine o listă goală: []. Copia de siguranță `inventar_istoric` rămâne intactă în memorie, păstrându-și toate elementele inițiale.

Implementarea în limbajul Python :

```
inventar_baza = ["Echipament", 50, 120, 35]
inventar_nou = [80, 15]
inventar_baza.remove("Echipament")
inventar_baza.extend(inventar_nou)
inventar_istoric = inventar_baza.copy()
inventar_baza.reverse()
inventar_baza.remove(120)
inventar_baza.clear()
print("Lista principală după golire totală:", inventar_baza)
print("Copia de siguranță (inventar_istoric):", inventar_istoric)
```

Testarea și verificarea pas cu pas

Pasul (Instrucțiunea executată)	Starea listei inventar_baza în memorie	Indecșii pozitivi curepți	Variabile create / Rezultate
Stare inițială	["Echipament", 50, 120, 35]	0: "Echipament", 1: 50, 2: 120, 3: 35	Alocare listă eterogenă
1. .remove("Echipament")	[50, 120, 35]	0: 50, 1: 120, 2: 35	Textul a fost eliminat din memorie
2. .extend(inventar_nou)	[50, 120, 35, 80, 15]	0: 50, 1: 120, 2: 35, 3: 80, 4: 15	Elementele din a doua listă au fost concatenate
3. .copy()	[50, 120, 35, 80, 15]	(Nemodificată structural)	inventar_istoric = [50, 120, 35, 80, 15]
4. .reverse()	[15, 80, 35, 120, 50]	0: 15, 1: 80, 2: 35, 3: 120, 4: 50	Ordinea elementelor inversată complet
5. .remove(120)	[15, 80, 35, 50]	0: 15, 1: 80, 2: 35, 3: 50	Valoarea 120 a fost extrasă din memorie
6. .clear()	[]	(Nu există indecși activi)	Toate elementele au fost eliminate

Afișare pe ecran:

Lista principală după golire totală: []

Copia de siguranță (inventar_istoric): [50, 120, 35, 80, 15]

Problemă propusă pentru tema de acasă

🎵 *Scenariu: „Aplicația de Streaming (Gestiunea avansată și Backup-ul Playlist-ului) – Faza II”*

Context: Ești dezvoltatorul unei aplicații moderne de muzică (similară cu Spotify). Pentru a asigura sincronizarea în cloud, serverul primește din baza de date o listă eterogenă brută a utilizatorului numită `playlist_baza`, care conține numele de cont premium și numărul de redări (vizualizări în milioane) ale melodiilor favorite, precum și o a doua listă cu recomandări noi numită `playlist_recomandat`. Pentru siguranța datelor, structurile trebuie fuzionate, salvate în backup, procesate și curățate după deconectare.

Enunț: Se dau inițial listele: `playlist_baza = ["Premium_User", 24, 85, 12]` și `playlist_recomandat = [40, 65]`.

Scrieți un program în limbajul Python care parcurge următoarele cerințe obligatorii (fiecare pas modifică direct lista rezultată de la pasul anterior):

- Curățarea inițială a datelor:** Eliminați din lista principală numele contului ("Premium_User") folosind metoda potrivită bazată pe valoare, astfel încât în colecție să rămână doar date pur numerice.
- Sincronizarea și unificarea listelor:** Utilizatorul dorește să adauge toate piesele din lista de recomandări în playlist-ul său. Extindeți lista `playlist_baza` prin adăugarea tuturor elementelor din `playlist_recomandat` într-o singură instrucțiune.
- Sistemul de siguranță (Backup):** Înainte de a schimba ordinea melodiilor, generați o copie complet independentă în memorie a playlist-ului extins într-o variabilă nouă numită `playlist_backup`, utilizând metoda de clonare.
- Inversarea ordinii de redare:** Pentru a asculta piesele în ordine inversă față de momentul adăugării, aplicați metoda de inversare structurală direct asupra listei `playlist_baza`.
- Eliminare melodie stornată:** Piesa care are exact 85 de milioane de redări a fost retrasă de pe platformă din cauza drepturilor de autor. Eliminați această valoare din lista principală inversată folosind metoda bazată pe valoare.
- Deconectare securizată (Golirea memoriei):** Utilizatorul închide aplicația. Ștergeți integral toate elementele din lista `playlist_baza` folosind metoda de curățare totală.
- Generare Raport:** Afișați pe ecran starea finală a listei principale, urmată de valorile din copia de siguranță precedate de mesajul exact: "Date salvate în backup pentru raport:".